```
# code...
require "pry"; binding.pry
# more code...
```

The call to binding.pry opens a Ruby interpreter in the current bindings, right inside the running process. From there, you can read and change your variables at will. When you want to exit the interpreter, just type exit to continue running the program. Thanks to this feature, Pry is a great alternative to traditional debuggers.

Pry is not the only command-line interpreter that uses bindings. Let's also look at irb, the default Ruby command-line utility.

The irb Example

At its core, irb is just a simple program that parses the standard input or a file and passes each line to eval. (This type of program is sometimes called a *Code Processor*.) Here's the line that calls eval, deep within irb's source code, in a file named workspace.rb:

Spell: Code Processor

```
eval(statements, @binding, file, line)
```

The statements argument is just a line of Ruby code. But what about those three additional arguments to eval? Let's go through them.

The first optional argument to eval is a Binding, and irb can change this argument to evaluate code in different contexts. This happens, for example, when you open a nested irb session on a specific object, by typing irb followed by the name of an object in an existing irb session. As a result, your next commands will be evaluated in the context of the object, similar to what instance_eval does.

What about file and line, the remaining two optional arguments to eval? These arguments are used to tweak the stack trace in case of exceptions. You can see how they work by writing a Ruby program that raises an exception:

ctwc/exception.rb

```
# this file raises an Exception on the second line
x = 1 / 0
```

You can process this program with irb by typing irb exception.rb at the prompt. If you do that, you'll get an exception on line 2 of exception.rb:

```
ZeroDivisionError: divided by 0
  from exception.rb:2:in `/'
```

When irb calls eval, it calls it with the filename and line number it's currently processing. That's why you get the right information in the exception's stack